Figure 10.16 Splitting a leaf along a non-optimal direction (in this case, the x axis is the direction with *minimal* variance). Comparing the final result to figure 10.14, although S_4's shape doesn't change significantly, S_{16} is more than doubled in size and it overlaps completely S_{17}: this means that any search targeted within S_{17} will also have to traverse S_{16}.

Figure 10.16 shows what happens when we need to split a leaf: first we recompute the radius and centroid of the leaf after including the new point, and then we also compute the variance of the M+1 points' coordinates along the k directions of the axis, in order to find the direction with highest variance; this is particularly useful with skewed sets of points, like S9 in the example, and helps reducing spheres volume and, in turn, overlap.

Listing 10.8 The SsNode::findClosestChild method

```
function
SsNode::findClosestChild(target)
       #1
  throw-if this.leaf
                                    #2
  minDistance ←
inf
#3
  result ←
null
    #4
  for each childNode in this.children
do                                          #5
    if distance(childNode.centroid, point) < minDistance
then                            #6
```